ICT 3212: Advanced Database Systems

Week 1: The Big Picture & Requirements Analysis
Session 1: Introduction to GlobalMart

Engr. Daniel MOUNE

moune.daniel@ictuniversity.edu.cm

ICT University, Cameroon Campus, Yaoundé
P.O. Box 526, 1 Avenue Dispensaire Messassi, Zoatoupsi

Imparting ICTs in all academic disciplines

February 24, 2026

Week 1 – Session 1: Course Overview & GlobalMart Design

Designing a complex e-commerce database from scratch

Building the foundation for advanced database features.

Today's Roadmap

Part 1: Course Overview (15 min)
- Why advanced databases?
- Course objectives and structure

Part 2: The GlobalMart Scenario (30 min)
- A complex e-commerce platform
- Domain description

Part 3: Requirements Analysis (30 min)
- Identifying entities and relationships
- Business rules

Part 4: Initial ERD Design (30 min)
- Brainstorm and sketch

BREAK (15 min)

Part 5: Lab Exercise (30 min)
- Draft ERD in teams

Part 6: Project Milestone (10 min)
- Form teams, brainstorm app ideas

"If you get lost, raise your hand – TAs are circulating."

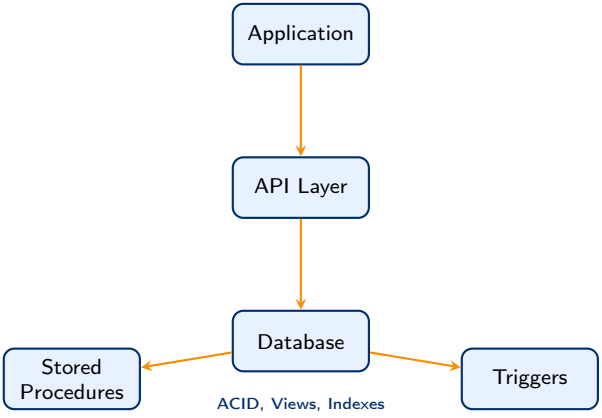
Why Advanced Database Systems?

The Limits of Simple CRUD

- Modern applications demand complex data logic.
- Business rules must be enforced at the database level.
- Concurrency, consistency, and performance are critical.

What We Will Master

- Stored procedures & triggers
- Views and materialized views
- Transaction isolation and locking
- Query optimization & indexing
- Database API design

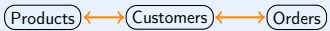


"We build systems that scale, remain consistent, and never lose data."

The GlobalMart Scenario – A Complex E-commerce Platform

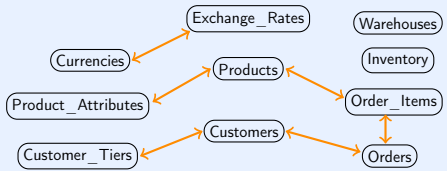
Typical E-commerce (Too Simple)

- Products, Customers, Orders
- Single currency
- One warehouse
- Fixed product attributes



GlobalMart (Our Challenge)

- Multi-currency + exchange rates
- Multiple warehouses with inventory
- Customer tiers (Standard, Premium, VIP)
- Variable product attributes (e.g., laptop RAM, shirt size)



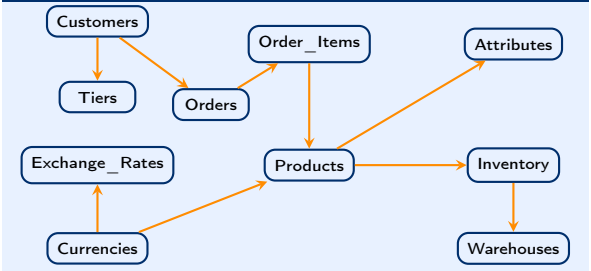
"GlobalMart will be our vehicle to master advanced database features."

Key Requirements & Initial ERD Sketch

Core Requirements

- Support multiple currencies with live exchange rates.
- Track inventory across several warehouses.
- Implement customer tiers with dynamic discounts.
- Handle products with varying attributes (e.g., electronics vs clothing).
- Ensure data integrity during high-volume sales (transactions).
- Automate reordering when stock is low (triggers).

Initial ERD Sketch (Entities & Relationships)

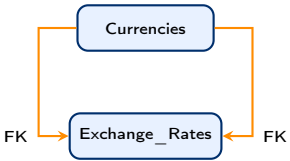


"We will refine this ERD throughout the session."

Deep Dive – Multi-Currency Support

The Challenge

GlobalMart sells to customers worldwide. Prices must be displayed and processed in multiple currencies, with exchange rates that fluctuate.



Core Tables

Currencies

- currency_code (PK): e.g., USD, EUR, GBP
- currency_name, symbol

Exchange_Rates

- from_currency, to_currency (FK to Currencies)
- rate: conversion factor (e.g., 1 USD = 0.85 EUR)
- effective_date: date when rate becomes active

Important

Always store order amount in both original currency and a base currency (e.g., USD) for consistent reporting.

"We will use the latest exchange rate when placing an order."

Deep Dive – Customer Tiers & Discounts

Tiered Loyalty Program

Customers are classified into tiers (Standard, Premium, VIP) based on their lifetime value. Each tier receives a different discount.

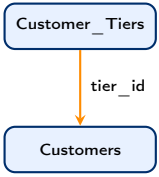
Tables

Customer_Tiers

- tier_id (PK)
- tier_name (e.g., 'VIP')
- discount_percentage (e.g., 15.0)
- min_lifetime_value

Customers

- customer_id (PK)
- name, email, ...
- tier_id (FK to Customer_Tiers)
- lifetime_value (computed)



Discount Application

When an order is placed, the discount from the customer's tier is applied to the subtotal.

"Tiers can be updated periodically based on purchase history."

Deep Dive – Product Attributes

The Problem

Different product categories have different attributes:

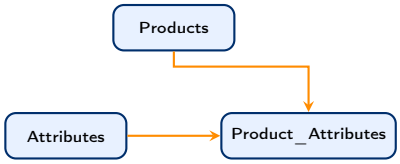
- Laptop: RAM, CPU, Screen size
- Shirt: Size, Color, Material

Fixed columns would lead to sparse tables and schema changes.

Entity-Attribute-Value (EAV) Approach

Products: product_id, name, base_price, category_id
Attributes: attribute_id, attribute_name (e.g., 'RAM')

Product_Attributes: product_id, attribute_id, value (e.g., '16GB')



Alternative: JSON Columns

Modern databases support JSON for semi-structured attributes. We'll explore both.

Lab Exercise: Draft Your ERD

Task (30 minutes)

In your teams, expand the initial ERD to include all entities and relationships we discussed:

- Customers, Customer_Tiers
- Products, Attributes, Product_Attributes
- Currencies, Exchange_Rates
- Warehouses, Inventory
- Orders, Order_Items

Instructions

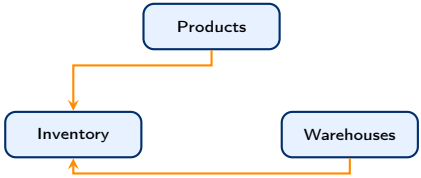
1. Use paper, whiteboard, or a digital tool (draw.io, Lucidchart).
2. For each entity, list at least 3 key attributes.
3. Define primary keys and foreign key relationships.
4. Note cardinalities (1:N, M:N).
5. Be prepared to explain your design choices to the class.

"Focus on capturing the core structure; we will refine it later."

Deep Dive – Inventory Management

Multi-Warehouse Challenge

GlobalMart operates multiple warehouses. Each product may be stored in several locations with different quantities.



Core Tables

Warehouses

- warehouse_id (PK), location, capacity

Inventory

- inventory_id (PK)
- product_id (FK to Products)
- warehouse_id (FK to Warehouses)
- quantity_on_hand
- reorder_threshold

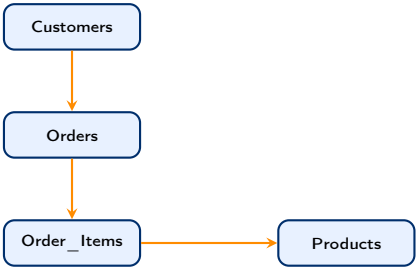
Business Rule

When `quantity_on_hand` falls below `reorder_threshold`, a trigger should automatically generate a reorder request.

Deep Dive – Orders and Order Items

Order Processing

An order is placed by a customer and contains multiple items. Each item references a specific product and quantity.



Tables

Orders

- order_id (PK)
- customer_id (FK)
- order_date
- total_amount (in base currency)
- status (pending, shipped, etc.)

Order_Items

- order_id (PK, FK)
- line_number (PK)
- product_id (FK)
- quantity, unit_price

"Line numbers ensure unique identification of each item within an order."

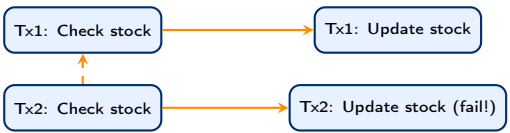
Transactional Integrity – Why ACID Matters

The "Last Item" Problem

Two customers try to buy the last unit of a popular product simultaneously. Without proper transaction handling, both orders might succeed, causing overselling.

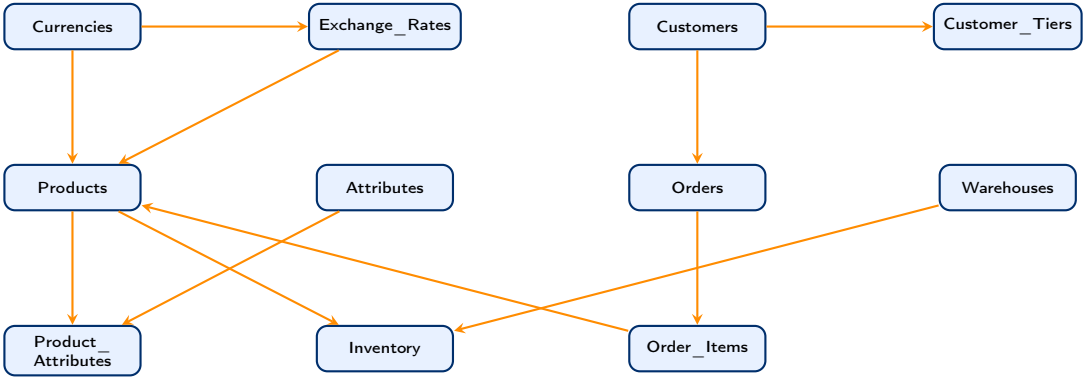
ACID Properties

- **Atomicity**: All parts of the order (create order, add items, update inventory) succeed or fail together.
- **Consistency**: Inventory never goes negative.
- **Isolation**: Concurrent orders don't interfere.
- **Durability**: Once committed, order is permanent.



"We will use transactions and isolation levels to prevent such anomalies."

Putting It All Together – Full GlobalMart Schema



"This is our target schema for the semester."

Project Milestone 1 – Team Formation & App Idea

Objective

Form teams of 3–4 and brainstorm a web application idea that could be built on top of a database like GlobalMart. The application and its database will be hosted online on free budget-friendly platforms. This will be your semester project.

Deliverables (Due next Monday)

- Team name and member list
- One-paragraph description of your app idea
- List of core entities and their relationships (initial sketch)
- Identify at least three advanced features you plan to implement (e.g., multi-currency, tiered discounts, inventory triggers)

Submission

Upload a single PDF (named `Milestone1_TeamX.pdf`) to the LMS by next Monday 23:59.

"This is your chance to build something impressive for your portfolio!"

Week 1 – Session 2: From ERD to DDL

Translating our GlobalMart design into actual SQL

Building the foundation – table by table.

Today's Roadmap

Part 1: ERD to Relational Mapping (20 min)
- Entities → Tables
- Relationships → Foreign Keys
- Attributes → Columns

Part 2: Data Types & Domains (25 min)
- Choosing the right types
- Domain constraints

Part 3: Constraints (30 min)
- Primary Keys, Foreign Keys
- CHECK, UNIQUE, NOT NULL

Part 4: Indexes (15 min)
- Why indexes?
- Where to create them

Part 5: Live Coding – DDL for GlobalMart (40 min)
- Writing CREATE TABLE scripts
- Adding sequences for order numbers

BREAK (15 min)

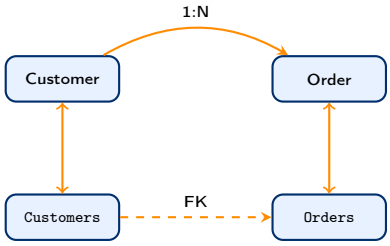
Part 6: Lab Exercise (30 min)
- Implement your team's schema

"We'll write SQL that will become the backbone of GlobalMart."

From ERD to Relational Schema

Rules

1. Each entity becomes a table.
2. Attributes become columns.
3. Primary key of entity becomes PK of table.
4. Relationships become foreign keys.
5. Many-to-many relationships become separate junction tables.



Example: Many-to-Many (Products ↔ Attributes)

Junction table `Product_Attributes` with composite PK (product_id, attribute_id).

Choosing Data Types – Best Practices

Key Considerations

- Use the smallest adequate type (saves space, improves performance).
- Prefer standard SQL types for portability.
- Consider domain constraints (e.g., currency amounts must be precise).

Numeric

- `INT` / `BIGINT` for IDs
- `DECIMAL(10,2)` for prices (exact)
- `SMALLINT` for quantity (if small)

Text

- `VARCHAR(n)` for variable strings
- `CHAR(n)` only for fixed length
- `TEXT` for long descriptions

Temporal

- `DATE` for dates
- `TIMESTAMP` for date+time
- `INTERVAL` for durations

GlobalMart Specific

`currency_code` → `CHAR(3)` (ISO codes)
`price` → `DECIMAL(12,2)`
`email` → `VARCHAR(255)` with CHECK for format

Implementing Constraints – PK, FK, CHECK, UNIQUE

Constraint Types

- **PRIMARY KEY**: Uniquely identifies each row.
- **FOREIGN KEY**: Links to another table (referential integrity).
- **UNIQUE**: No duplicate values.
- **CHECK**: Enforces a condition on column values.
- **NOT NULL**: Column must have a value.

GlobalMart Examples

```
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    tier_id INT REFERENCES Customer_Tiers(
        tier_id),
    lifetime_value DECIMAL(12,2) DEFAULT 0.0,
    CHECK (email ~* '^[A-Za-z0-9._\%-]+@[A-Za-
        z0-9.-]+[.][A-Za-z]+$')
);
```

"Constraints are the guardians of data integrity."

Indexes – Why and Where

What is an Index?

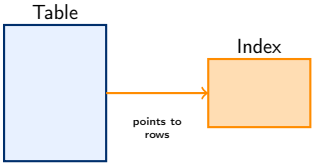
database index is a data structure (like a B-Tree) that improves the speed of data retrieval operations on a table at the cost of additional writes and storage space.

When to Create Indexes

- Primary keys (automatically indexed)
- Foreign keys (to speed up joins)
- Columns used frequently in `WHERE`, `ORDER BY`, `GROUP BY`
- Columns with high selectivity (many unique values)

GlobalMart Index Candidates

- `Customers.email` (unique, used for login)
- `Orders.customer_id` (FK)
- `Order_Items.product_id` (FK)
- `Inventory.warehouse_id` (FK)
- `Exchange_Rates.effective_date` (range queries)



"Indexes speed up reads but slow down writes – choose wisely!"

Live Coding – Creating GlobalMart Tables (1)

Step 1: Create Database and Connect

```
CREATE DATABASE GlobalMart;
\c GlobalMart  -- or USE GlobalMart;
```

Step 2: Create Independent Tables

```
CREATE TABLE Currencies (
    currency_code CHAR(3) PRIMARY KEY,
    currency_name VARCHAR(50) NOT NULL,
    symbol VARCHAR(10)
);

CREATE TABLE Customer_Tiers (
    tier_id INT PRIMARY KEY,
    tier_name VARCHAR(20) NOT NULL,
    discount_percentage DECIMAL(5,2) CHECK (discount_percentage BETWEEN 0 AND 100),
    min_lifetime_value DECIMAL(12,2) DEFAULT 0
);

CREATE TABLE Warehouses (
    warehouse_id INT PRIMARY KEY,
    location VARCHAR(100) NOT NULL,
    capacity INT
);
```

Live Coding – Tables with Foreign Keys (2)

Step 3: Tables that Reference Others

```
CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    tier_id INT REFERENCES Customer_Tiers(tier_id),
    lifetime_value DECIMAL(12,2) DEFAULT 0.0,
    registered_date DATE DEFAULT CURRENT_DATE
);
CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(200) NOT NULL,
    base_price DECIMAL(12,2) NOT NULL,
    currency_code CHAR(3) REFERENCES Currencies(currency_code),
    category VARCHAR(50)
);
CREATE TABLE Attributes (
    attribute_id INT PRIMARY KEY,
    attribute_name VARCHAR(50) NOT NULL
);
```


"Notice the use of `REFERENCES` shorthand for foreign keys."

Live Coding – Junction Tables and Sequences (3)

Step 4: Many-to-Many and Inventory

```
CREATE TABLE Product_Attributes (
    product_id INT REFERENCES Products(
        product_id),
    attribute_id INT REFERENCES
        Attributes(attribute_id),
    value VARCHAR(100) NOT NULL,
    PRIMARY KEY (product_id, attribute_id
        )
);

CREATE TABLE Inventory (
    inventory_id INT PRIMARY KEY,
    product_id INT REFERENCES Products(
        product_id),
    warehouse_id INT REFERENCES
        Warehouses(warehouse_id),
    quantity_on_hand INT DEFAULT 0,
    reorder_threshold INT DEFAULT 10,
    UNIQUE(product_id, warehouse_id)
);
```

Step 5: Sequences for Auto-Increment

```
CREATE SEQUENCE order_seq START 1000;

CREATE TABLE Orders (
    order_id INT PRIMARY KEY DEFAULT nextval('order_seq
        '),
    customer_id INT REFERENCES Customers(customer_id),
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(12,2),
    status VARCHAR(20) DEFAULT 'PENDING'
);
```

Break Time – 15 Minutes

Time to go on PostgreSQL and practice.

"Try creating the tables we just coded, and experiment with inserting some sample data."

Lab Exercise – Implement Your Team's Schema

Task (30 minutes)

Using the SQL we just wrote as a guide, create the complete schema for your team's project idea.

- Write `CREATE TABLE` statements for all entities.
- Include primary keys, foreign keys, and appropriate constraints.
- Add indexes on foreign key columns.
- Use sequences for auto-incrementing IDs if needed.

Instructions

1. Work in your project teams.
2. Use a shared online editor (e.g., SQL Fiddle, DB Fiddle) or local database.
3. Execute your script and verify that all tables are created.
4. Take a screenshot or copy the output for submission.

"If you have not yet completed installation of PostgreSQL, please do so before the next class."

Class 02 Summary Next Steps

What We Covered Today

- ERD to relational mapping
- Choosing appropriate data types
- Implementing constraints (PK, FK, CHECK, UNIQUE)
- Creating indexes for performance
- Writing DDL scripts for GlobalMart

Before Next Class

- Finalize your team's schema and submit the SQL script.
- Read: Chapter 3 on advanced SQL queries (joins, subqueries).
- Next class: We'll populate the database with sample data and write complex queries.

Questions?

Office hours: Wednesdays 2–4 PM, or email / LMS

Week 3 - Populating the World

Inserting realistic data and writing advanced queries

From raw data to business intelligence.

Today's Roadmap

Part 1: Populating Tables (20 min)
- Sample data for GlobalMart
- INSERT statements

Part 2: Complex JOINs (25 min)
- Customer order history
- Multi-table joins

Part 3: Subqueries (25 min)
- Correlated vs. non-correlated
- Top-selling products

Part 4: Window Functions (30 min)
- RANK, DENSE_RANK
- LAG, LEAD

BREAK (15 min)

Part 5: Lab Exercise (30 min)
- Sales report by category and week

"We will write queries that will become the foundation of our reports and dashboards."

Populating GlobalMart - Sample Data

Inserting Customers

```
INSERT INTO Customers (customer_id, name, email, tier_id, lifetime_value)
VALUES
  (1, 'Alice Johnson', 'alice@example.com', 1, 5000.00),
  (2, 'Bob Smith', 'bob@example.com', 2, 15000.00),
  (3, 'Carol White', 'carol@example.com', 3, 25000.00);
```

Inserting Orders and Order Items

```
INSERT INTO Orders (order_id, customer_id, order_date, total_amount, status)
VALUES
  (101, 1, '2025-02-15', 120.00, 'COMPLETED'),
  (102, 2, '2025-02-20', 250.00, 'COMPLETED');

INSERT INTO Order_Items (order_id, line_number, product_id, quantity, unit_price)
VALUES
  (101, 1, 1001, 2, 60.00),
  (102, 1, 1002, 1, 250.00);
```

"We will use larger datasets for realistic exercises."

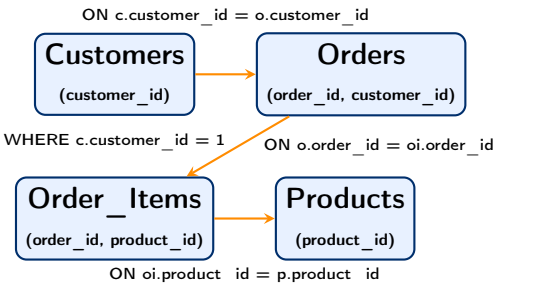
Complex JOINs - Customer Order History

Query Goal

For a given customer (e.g., Alice), retrieve all their orders with product names and quantities.

SQL with Multiple JOINs

```
SELECT
    c.name AS customer_name,
    o.order_id,
    o.order_date,
    p.product_name,
    oi.quantity,
    oi.unit_price,
    (oi.quantity * oi.unit_price) AS
        line_total
FROM
    Customers c
    JOIN Orders o ON c.customer_id = o.
        customer_id
    JOIN Order_Items oi ON o.order_id = oi.
        order_id
    JOIN Products p ON oi.product_id = p.
        product_id
WHERE
    c.customer_id = 1
ORDER BY
    o.order_date DESC, oi.line_number;
```



"Visualise the joins: Customers → Orders → Order_Items → Products."

Subqueries - Top 5 Best-Selling Products

Using a Subquery to Rank

Find the top 5 products by total quantity sold.

Subquery in ORDER BY / LIMIT

```
SELECT
    p.product_id,
    p.product_name,
    SUM(oi.quantity) AS total_sold
FROM
    Products p
    JOIN Order_Items oi
        ON p.product_id = oi.product_id
GROUP BY
    p.product_id, p.product_name
ORDER BY
    total_sold DESC
LIMIT 5;
```

Correlated Subquery Example

```
-- Find products whose sales
-- exceed the average sales
-- of all products
SELECT product_name, total_sold
FROM (
    SELECT p.product_name,
        SUM(oi.quantity) AS total_sold
    FROM Products p JOIN Order_Items oi
        ON ...
    GROUP BY p.product_name
) AS product_totals
WHERE total_sold >
(SELECT AVG(total_sold) FROM (...));
```

Window Function - RANK()

Purpose

Assigns a rank to each row within a partition. Rows with equal values receive the same rank, and the next rank is skipped (e.g., 1,2,2,4).

Example: Top-spending customer per tier

```
SELECT
    customer_id,
    name,
    tier_id,
    lifetime_value,
    RANK() OVER (PARTITION BY tier_id ORDER BY lifetime_value DESC) AS rank_in_tier
FROM Customers;
```

Sample Output

customer_id	name	tier_id	lifetime_value	rank_in_tier
101	Alice	1	5000	1
102	Bob	1	3000	2
201	Carol	2	15000	1
202	Dave	2	15000	1
203	Eve	2	12000	3

Carol and Dave both rank 1; next rank is 3 (skipped).

Window Function - LAG()

Purpose

Accesses data from a **previous** row in the same result set without a self-join. Useful for comparing current value with a previous value (e.g., price change over time).

Example: Track product price history

```
SELECT
    product_id,
    change_date,
    new_price,
    LAG(new_price, 1) OVER (PARTITION BY product_id ORDER BY change_date) AS previous_price,
    new_price - LAG(new_price, 1) OVER (...) AS price_change
FROM Price_History;
```

Sample Output (product_id = 1001)

change_date	new_price	previous_price	price_change
2025-01-01	100.00	NULL	NULL
2025-02-01	110.00	100.00	10.00
2025-03-01	105.00	110.00	-5.00

First row has no previous price → NULL.

Window Function - LEAD()

Purpose

Accesses data from a **subsequent** row in the same result set. Useful for comparing current value with a future value (e.g., predicting next price).

Example: Show current and next price

```
SELECT
    product_id,
    change_date,
    new_price,
    LEAD(new_price, 1) OVER (PARTITION BY product_id ORDER BY change_date) AS next_price
FROM Price_History;
```

Sample Output (product_id = 1001)

change_date	new_price	next_price
2025-01-01	100.00	110.00
2025-02-01	110.00	105.00
2025-03-01	105.00	NULL

Last row has no next price → NULL.

Window Function - ROW_NUMBER()

Purpose

Assigns a unique sequential integer to each row within a partition, starting at 1. Unlike `RANK()`, ties are broken arbitrarily (or by a secondary `ORDER BY`).

Example: First order per customer (earliest date)

```
SELECT customer_id, order_id, order_date
FROM (
    SELECT
        customer_id, order_id, order_date,
        ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS rn
    FROM Orders
) ranked
WHERE rn = 1;
```

Output

customer_id	order_id	order_date
1	101	2025-01-15
2	105	2025-01-20
3	110	2025-01-22

Each customer gets exactly one row (their earliest order).

Window Function - DENSE_RANK()

Difference from RANK()

`DENSE_RANK()` assigns consecutive ranks without gaps. If two rows tie for rank 1, the next rank is 2 (not 3).

Example: Compare `RANK()` and `DENSE_RANK()`

```
SELECT
    score,
    RANK() OVER (ORDER BY score DESC) AS rank,
    DENSE_RANK() OVER (ORDER BY score DESC) AS dense_rank
FROM Scores;
```

Output

score	rank	dense_rank
100	1	1
100	1	1
95	3	2
90	4	3

Use `RANK()` when you want gaps to reflect count of previous rows; use `DENSE_RANK()` when you want consecutive integers.

Window Function - NTILE()

Purpose

Divides rows into a specified number of approximately equal groups (buckets). Useful for percentiles, quartiles, or deciles.

Example: Group customers into 4 quartiles by lifetime value

```
SELECT
    customer_id,
    name,
    lifetime_value,
    NTILE(4) OVER (ORDER BY lifetime_value DESC) AS quartile
FROM Customers;
```

Output (sample)

customer_id	name	lifetime_value	quartile
203	Eve	25000	1
201	Carol	15000	1
202	Dave	12000	2
101	Alice	5000	3
102	Bob	3000	4

Top 25% get quartile 1, next 25% quartile 2, etc.

Window Functions - FIRST_VALUE() and LAST_VALUE()

Purpose

Return the value from the first or last row in a window frame.

Example: For each product, show the earliest price and latest price

```
SELECT DISTINCT
    product_id,
    FIRST_VALUE(new_price) OVER (PARTITION BY product_id ORDER BY change_date) AS first_price,
    LAST_VALUE(new_price) OVER (PARTITION BY product_id ORDER BY change_date
                                ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_price
FROM Price_History;
```

Note on LAST_VALUE

By default, the frame is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. To get the last value of the whole partition, you must specify `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.

Lab Exercise - Weekly Sales Report

Task (30 minutes)

Write a query that generates a sales report showing:

- Total revenue (in USD) per product category.
- Broken down by week (using the order date).
- For the last month (e.g., February 2025).

Use the tables: `Orders`, `Order_Items`, `Products`, `Categories`.

Expected Output Columns

```
category_name, year_week, total_revenue
```

Hints

- Use `DATE_TRUNC('week', order_date)` to group by week.
- Join all four tables.
- Filter `order_date >= '2025-02-01' AND order_date < '2025-03-01'`.
- Use `SUM(oi.quantity * oi.unit_price)`.

"Submit your SQL query and a screenshot of the results."

Bulk Data Insertion - Populating Efficiently

Inserting Many Rows

Instead of many single ‘INSERT‘ statements, use:

- Multi-row ‘INSERT‘ (SQL standard)
- ‘INSERT ... SELECT‘ (copy from another table)
- ‘COPY‘ command (from CSV, fastest)

Examples

```
-- Multi-row INSERT
INSERT INTO Products (product_id, name, price) VALUES
(1, 'Laptop', 1000),
(2, 'Mouse', 25),
(3, 'Keyboard', 75);

-- INSERT ... SELECT (generate test data)
INSERT INTO Order_Items (order_id, line_number, product_id, quantity)
SELECT order_id, row_number() OVER (PARTITION BY order_id), product_id, 1
FROM (SELECT order_id, unnest(array[101,102]) AS product_id FROM Orders) t;

-- COPY from CSV file
COPY Customers FROM '/data/customers.csv' DELIMITER ',' CSV HEADER;
```

Common Table Expressions (CTEs)

Purpose

CTEs create temporary named result sets that can be referenced within a ‘SELECT‘, ‘INSERT‘, ‘UPDATE‘, or ‘DELETE‘. They improve readability and enable recursion.

Example: Monthly sales per category using CTE

```
WITH monthly_category_sales AS (
    SELECT
        c.category_name,
        DATE_TRUNC('month', o.order_date) AS month,
        SUM(oi.quantity * oi.unit_price) AS revenue
    FROM Orders o
    JOIN Order_Items oi ON o.order_id = oi.order_id
    JOIN Products p ON oi.product_id = p.product_id
    JOIN Categories c ON p.category_id = c.category_id
    GROUP BY c.category_name, DATE_TRUNC('month', o.order_date)
)
SELECT * FROM monthly_category_sales
WHERE revenue > 1000
ORDER BY month, revenue DESC;
```

Lab Exercise - Window Functions Practice

Task (20 minutes)

Write queries using window functions to answer:

1. For each product, rank its sales (by total quantity) within its category. Use ‘RANK()‘.
2. For each customer, show their order amount and the amount of the previous order (‘LAG()‘).
3. Split customers into 10 deciles based on lifetime value (‘NTILE(10)‘).

Sample Query for Task 1

```
SELECT
    p.product_name,
    c.category_name,
    SUM(oi.quantity) AS total_sold,
    RANK() OVER (PARTITION BY c.category_id ORDER BY SUM(oi.quantity) DESC) AS rank_in_category
FROM Products p
JOIN Order_Items oi ON p.product_id = oi.product_id
JOIN Categories c ON p.category_id = c.category_id
GROUP BY p.product_id, p.product_name, c.category_id, c.category_name;
```

Project Milestone 1 - Reminder

Due End of This Week

Team formation and app idea submission.

What to Submit

- Team name and member list.
- One-paragraph description of your app idea.
- List of core entities and their relationships (initial ERD sketch).
- Identify at least three advanced features you plan to implement (e.g., multi-currency, tiered discounts, inventory triggers, full-text search).

"This is the foundation of your final project - be creative and realistic."

Week 3 Summary Next Steps

What We Covered Today

- Populated GlobalMart tables with sample data.
- Wrote complex JOIN queries for order history.
- Used subqueries to find top-selling products.
- Introduced window functions (RANK, LAG).
- Lab: weekly sales report by category.

Before Next Class

- Complete the weekly sales report query.
- Submit Project Milestone 1.
- Read: Chapter 2 on stored procedures (beginning).
- Next class: We start building business logic with stored procedures!

Questions?

Office hours: Wednesdays 2-4 PM, or email / LMS